

Universidade da Beira Interior

Departamento de Informática



**Departamento de
Informática**

Sistemas Distribuídos: AutoUBI

Elaborado por:

TIAGO LEANDRO OLIVEIRA VALENTE a48234

RITA FRAZÃO DE OLIVEIRA GOMES a49519

TIAGO GONÇALVES PINHO DA SILVA MARQUES a48950

Orientadores:

Professor Tiago Simões

Professor Nuno Pereira

24 de maio de 2026

Conteúdo

Conteúdo	i
1 Introdução	1
1.1 Motivação	1
1.2 Objetivos	2
1.3 Estrutura do Documento	3
2 Estado da Arte	5
2.1 Arquitetura Web Multicamadas	5
2.2 Spring Boot, MVC e Endpoints	5
2.3 Persistência com JPA e Hibernate	6
2.4 Segurança em Aplicações Web	6
2.5 Conclusões	6
3 Arquitetura e Modelo do Sistema	7
3.1 Arquitetura do Sistema	7
3.2 Modelo de Dados	8
3.3 Endpoints e Responsabilidades	8
3.4 Fluxo de Compra	9
3.5 Conclusões	9
4 Implementação e Testes	11
4.1 Entidades Principais	11
4.2 Spring Security	12
4.2.1 Autenticação	12
4.2.2 Autorização	13
4.2.3 CSRF	13
4.2.4 Logout	13
4.3 Carrinho de Visitante e Migração de Sessão	13
4.4 Checkout e Faturas	14
4.5 Validação de Dados	14
4.6 Testes	15
4.7 Limitações	15

4.8 Conclusões	16
5 Conclusão	17
Bibliografia	19

Acrónimos

AJAX	<i>Asynchronous JavaScript and XML</i>
API	<i>Application Programming Interface</i>
CSRF	<i>Cross-Site Request Forgery</i>
HTML	<i>HyperText Markup Language</i>
HTTP	<i>HyperText Transfer Protocol</i>
JPA	<i>Java Persistence API</i>
JSON	<i>JavaScript Object Notation</i>
MVC	<i>Model-View-Controller</i>
PDF	<i>Portable Document Format</i>
REST	<i>Representational State Transfer</i>
SQL	<i>Structured Query Language</i>
XSS	<i>Cross-Site Scripting</i>

Capítulo

1

Introdução

O presente projeto surge no contexto da unidade curricular de Sistemas Distribuídos e consiste no desenvolvimento de uma aplicação web para uma loja online de automóveis, denominada **AutoUBI**. A aplicação foi implementada em Java, com Spring Boot, Spring MVC, Spring Security e persistência em MariaDB/MySQL através de Spring Data *Java Persistence API* (JPA).

O sistema permite a visualização de um catálogo de veículos organizado por categorias, a gestão de produtos e stock, a realização de compras online, a emissão de faturas em *Portable Document Format* (PDF) e o acompanhamento do estado das encomendas. Foram ainda implementados mecanismos de autenticação, autorização e gestão de sessões, permitindo distinguir entre administradores, clientes autenticados e visitantes.

1.1 Motivação

A principal motivação para este trabalho resulta da necessidade de aplicar conceitos de sistemas distribuídos num cenário mais próximo de uma aplicação real. Uma loja online é um bom caso de estudo porque obriga a lidar com utilizadores, permissões, validação de dados, atualização de stock, geração de documentos e comunicação contínua entre browser, servidor e base de dados.

Além disso, o projeto permitiu consolidar conceitos de aplicações web modernas. Entre eles destacam-se a arquitetura MVC, os serviços REST, a gestão de sessões, a segurança com Spring Security e os pedidos AJAX.

1.2 Objetivos

De forma a concretizar o projeto, a aplicação foi concebida segundo uma arquitetura multicamadas, em que os controladores Spring recebem os pedidos *HyperText Transfer Protocol* (HTTP), a camada de negócio valida e processa as operações e a camada de persistência comunica com a base de dados relacional.

Mais especificamente, este projeto visa:

1. Implementar uma aplicação web completa com arquitetura *Model-View-Controller* (MVC) sobre Spring Boot;
2. Disponibilizar um catálogo de veículos organizado por categorias;
3. Permitir a gestão de produtos, incluindo criação, edição, remoção, consulta, imagem, stock e ficha técnica;
4. Gerir utilizadores com autenticação baseada em base de dados e diferentes níveis de permissão, nomeadamente *ADMIN* e *USER*;
5. Suportar navegação e compras sem conta através de um carrinho de visitante guardado em sessão;
6. Migrar automaticamente o carrinho de visitante aquando do registo ou *login*;
7. Implementar o fluxo completo de compra: catálogo → carrinho → *checkout* → fatura PDF;
8. Disponibilizar um painel de administração com estatísticas, gestão de clientes, histórico de compras e gestão de encomendas;
9. Garantir aspetos de segurança, incluindo proteção *Cross-Site Request Forgery* (CSRF), validação de dados, palavras-passe com BCrypt e controlo de sessões;
10. Permitir ao cliente acompanhar o estado das suas encomendas e descarregar a respetiva fatura.

Pretende-se, assim, apresentar uma aplicação funcional e coerente com os objetivos da unidade curricular. Ao mesmo tempo, reconhece-se que algumas opções, como `ddl-auto=update` e paginação no cliente, são adequadas ao contexto académico, mas exigiriam revisão num cenário de produção.

1.3 Estrutura do Documento

O presente relatório encontra-se organizado em cinco capítulos. O primeiro introduz o projeto e os objetivos. O segundo relaciona as tecnologias escolhidas com decisões concretas. O terceiro descreve arquitetura, modelo de dados e endpoints. O quarto detalha implementação, segurança, validação e testes. O quinto apresenta conclusões e trabalho futuro.

Capítulo

2

Estado da Arte

Este capítulo apresenta os conceitos que influenciaram diretamente a implementação do AutoUBI, relacionando cada tecnologia com uma decisão concreta do projeto.

2.1 Arquitetura Web Multicamadas

A separação por camadas foi adotada para evitar que os templates Thymeleaf acessassem diretamente à base de dados ou que os controladores concentrassem toda a lógica da aplicação. Esta decisão segue a ideia de dividir um sistema distribuído em componentes com responsabilidades claras e comunicação bem definida [1, 2].

No AutoUBI, esta separação ficou organizada em três níveis principais: vistas Thymeleaf no frontend, controladores Spring MVC para tratar pedidos HTTP e repositórios Spring Data JPA para persistência. A vantagem prática foi tornar mais simples alterar, por exemplo, a página do catálogo sem modificar o modelo de dados, ou ajustar uma consulta sem alterar a navegação do utilizador.

2.2 Spring Boot, MVC e Endpoints

A arquitetura MVC foi escolhida porque o projeto exigia páginas renderizadas no servidor, formulários tradicionais e algumas interações assíncronas. Uma alternativa possível teria sido criar uma *Application Programming Interface* (API) separada e um frontend em React ou Vue, mas essa opção aumentaria a complexidade sem trazer ganhos relevantes para os requisitos definidos.

Os controladores Spring centralizam a entrada dos pedidos e fazem a ligação entre a vista e a camada de dados. Esta abordagem foi particularmente útil no carrinho: algumas ações usam navegação normal, enquanto outras atualizam apenas o contador ou a quantidade de itens, evitando recarregar a página inteira.

2.3 Persistência com JPA e Hibernate

A persistência foi implementada com JPA e Hibernate para reduzir a escrita manual de *Structured Query Language* (SQL) e manter o modelo próximo das classes Java. Esta escolha facilitou relações entre veículos, categorias, faturas e itens de fatura, e simplificou a geração da fatura e do histórico de compras.

Uma limitação assumida foi a utilização de `ddl-auto=update`. Esta opção acelerou o desenvolvimento, mas não é adequada para produção, porque as alterações ao esquema deixam de ser versionadas. Por esse motivo, a migração para Flyway ou Liquibase é identificada como melhoria futura.

2.4 Segurança em Aplicações Web

Como a aplicação gere contas, compras e encomendas, a segurança não podia ficar limitada ao formulário de *login*. Foram aplicadas autenticação com base de dados, palavras-passe com BCrypt, autorização por *roles*, proteção CSRF e invalidação correta de sessão no *logout*. Estes aspetos relacionam-se com os conteúdos sobre desenho de sistemas robustos e proteção de interações distribuídas [3, 4, 5].

A decisão mais relevante foi distinguir claramente três perfis: visitante, cliente autenticado e administrador. Esta separação obrigou a tratar casos reais, como carrinho antes do *login*, conta suspensa e tentativa de acesso indevido ao painel de administração.

2.5 Conclusões

As tecnologias escolhidas foram adequadas ao âmbito do projeto, mas não são neutras: Spring Boot simplificou a configuração, Hibernate acelerou a persistência e Thymeleaf reduziu a complexidade do frontend. Assim sendo, a solução fica mais dependente do servidor para renderização de páginas e ainda carece de migrações formais de base de dados para um contexto de produção.

Capítulo

3

Arquitetura e Modelo do Sistema

Este capítulo descreve a organização global do AutoUBI. O objetivo é mostrar como os pedidos percorrem o sistema, que dados são persistidos e que endpoints sustentam as principais funcionalidades.

3.1 Arquitetura do Sistema

A aplicação foi implementada com uma organização MVC. O browser envia pedidos HTTP, os controladores Spring tratam a navegação e a validação inicial, os repositórios encapsulam o acesso aos dados e a base de dados MariaDB guarda o estado persistente. Esta separação foi importante para evitar controladores demasiado dependentes de detalhes de SQL e para manter os templates focados na apresentação.

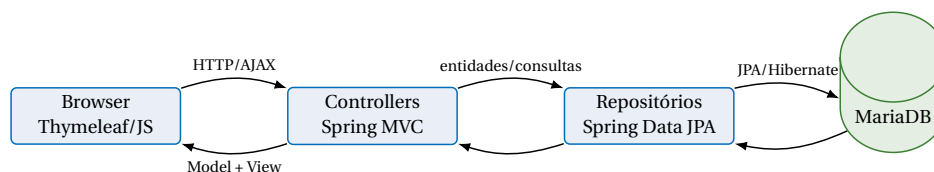


Figura 3.1: Fluxo de pedido na arquitetura MVC do AutoUBI

Na prática, nem todos os pedidos seguem exatamente o mesmo caminho. A consulta do catálogo apenas lê veículos e categorias; já o *checkout* valida stock, cria a fatura, cria os itens da fatura, atualiza quantidades e limpa o carrinho. Por isso, esta operação foi tratada como o fluxo mais crítico do sistema.

3.2 Modelo de Dados

O modelo de dados foi desenhado em torno de seis entidades principais. A entidade de cliente representa clientes e administradores. Veículos e categorias estruturam o catálogo. O carrinho persistente é guardado numa entidade própria. As compras concluídas ficam registadas em faturas e itens de fatura; o campo estado permite acompanhar a encomenda após a compra.

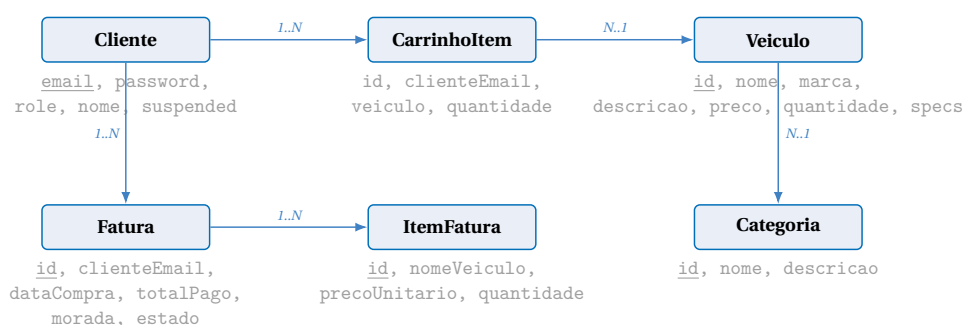


Figura 3.2: Diagrama simplificado das entidades e relações

A decisão de guardar a ficha técnica do veículo no campo specs, em formato *JavaScript Object Notation* (JSON), teve uma vantagem e uma desvantagem. A vantagem foi permitir atributos técnicos variáveis sem alterar o esquema relacional. A desvantagem é que filtros complexos diretamente na base de dados ficam menos simples do que seriam com colunas próprias.

3.3 Endpoints e Responsabilidades

A Tabela 3.1 resume as rotas principais. Esta visão é mais útil do que listar apenas nomes de classes, porque mostra que funcionalidades são públicas, quais exigem autenticação e quais ficam restritas ao administrador.

Rota	Responsabilidade	Acesso
/cliente	Catálogo e filtros de veículos	Público
/cliente/veiculo/id	Detalhe de veículo	Público
/guest/**	Carrinho de visitante guardado em sessão	Público
/cliente/carrinho/**	Carrinho autenticado e confirmação de compra	Cliente
/cliente/carrinho/checar/estado	Verificação de estado, método de pagamento e validação de stock	Cliente
/api/fatura/**	Geração e download de fatura em PDF	Cliente
/perfil	Dados pessoais, estatísticas e encomendas	Cliente
/admin/**	Inventário, clientes, compras, estatísticas e encomendas	Admin

Tabela 3.1: Principais endpoints e níveis de acesso

3.4 Fluxo de Compra

O fluxo mais sensível é o *checkout*. O sistema começa por ler os itens do carrinho, confirma se ainda existe stock suficiente, desconta as unidades em Veiculo e acumula o total, cria e guarda a Fatura, associa os ItemFatura e limpa o carrinho. Esta ordem garante que o total é calculado com os preços atuais antes de a fatura ser persistida.

No caso de visitante, o carrinho existe apenas em sessão. Após *login* ou registo, os itens são migrados para a tabela `carrinho_item`. Esta decisão resolveu um problema prático: permitir navegação livre antes da autenticação sem perder os produtos escolhidos.

3.5 Conclusões

A arquitetura final ficou adequada para uma aplicação web académica: simples, modular e suficientemente segura. O ponto mais frágil é a ausência de uma camada de serviços claramente separada em todas as operações, o que pode tornar alguns controladores demasiado extensos caso o projeto cresça.

Capítulo

4

Implementação e Testes

Este capítulo descreve os mecanismos principais do AutoUBI: entidades, segurança, gestão de sessão, validação de dados e testes.

4.1 Entidades Principais

As entidades seguintes representam os dados essenciais da aplicação: utilizadores, veículos e compras concluídas.

```
@Entity @Table(name = "cliente")
public class Cliente {
    @Id
    private String email;
    private String password; // BCrypt
    private String role;     // USER | ADMIN
    private String nome;
    private boolean suspended; }

@Entity @Table(name = "veiculo")
public class Veiculo {
    @Id @GeneratedValue
    private Long id;
    private String nome, marca, imagem;
    private Integer ano;
    private String descricao;
    private double preco;
    private int quantidade;
    @Column(columnDefinition = "TEXT")
    private String specs; // ficha tecnica em JSON
    @ManyToOne
    private Categoria categoria; }
```

Excerto de Código 4.1: Entidades Cliente e Veiculo (campos relevantes)

A entidade `Cliente` concentra os dados usados na autenticação e autorização. A entidade `Veiculo` guarda o stock, a categoria e a ficha técnica usada no catálogo.

```
@Entity @Table(name = "fatura")
public class Fatura {
    @Id @GeneratedValue
    private Long id;
    private String clienteEmail;
    private LocalDateTime dataCompra;
    private double totalPago;
    private String metodoPagamento, morada;
    private String estado = "Pendente";
    @OneToMany(mappedBy = "fatura", cascade = ALL)
    private List<ItemFatura> itens;
}

@Entity @Table(name = "item_fatura")
public class ItemFatura {
    @Id @GeneratedValue
    private Long id;
    @ManyToOne
    private Fatura fatura;
    private String nomeVeiculo;
    private double precoUnitario;
    private int quantidade;
}
```

Excerto de Código 4.2: Entidades Fatura e ItemFatura (campos relevantes)

A separação entre `Fatura` e `ItemFatura` preserva o histórico da compra, mesmo que os dados do veículo sejam alterados posteriormente.

4.2 Spring Security

A segurança da aplicação é configurada em `SecurityConfig` e assenta nos seguintes mecanismos.

4.2.1 Autenticação

- **Provider:** `DaoAuthenticationProvider` ligado ao serviço que carrega o utilizador por e-mail.

- **Palavras-passe:** são armazenadas com BCrypt, usando um encoder configurado no Spring Security.
- **Sessões:** após autenticação, é criada uma sessão para associar as ações do utilizador à sua conta. No *logout*, a sessão é invalidada.
- **Contas suspensas:** quando `cliente.isSuspended()` é verdadeiro, o `UserDetails` devolvido fica desativado, bloqueando o *login*.

4.2.2 Autorização

- Rotas `/admin/**` — apenas a autoridade ADMIN.
- Rotas de cliente, perfil, *checkout* e faturas — apenas a autoridade USER.
- Rotas de visitante, catálogo, *login*, registo e recursos estáticos — acesso público.

4.2.3 CSRF

A proteção CSRF está ativa nos formulários. Nos pedidos *Asynchronous JavaScript and XML* (AJAX), o token é lido de meta-tags no cabeçalho.

```
<meta name="_csrf" th:content="${_csrf.token}"/>
<meta name="_csrf_header" th:content="${_csrf.headerName}"/>
```

Excerto de Código 4.3: Meta-tags CSRF para pedidos AJAX

4.2.4 Logout

O *logout* é realizado via **POST**, mantendo a proteção CSRF, e redireciona para `/login?logout=true`.

4.3 Carrinho de Visitante e Migração de Sessão

O carrinho de visitante é gerido por `GuestCarrinhoController` e armazenado na sessão HTTP como um mapa entre ID do veículo e quantidade.

Após *login* ou registo bem-sucedido, o `AuthenticationSuccessHandler`:

1. Lê o carrinho de sessão do visitante.
2. Persiste cada item na tabela `carrinho_item`, associando-o ao e-mail do utilizador autenticado.

3. Remove o carrinho temporário da sessão.
4. Redireciona contas com autoridade ADMIN para /admin; contas USER são redirecionadas para /cliente/carrinho se havia itens migrados, ou para /cliente caso contrário.

Esta migração evita que o utilizador perca produtos escolhidos antes de iniciar sessão.

4.4 Checkout e Faturas

No *checkout*, o sistema executa os seguintes passos:

1. Lê os itens do carrinho.
2. Confirma se existe stock suficiente.
3. Desconta o stock de cada Veiculo e acumula o total.
4. Cria e guarda a Fatura com morada, método de pagamento, total calculado e estado inicial *Pendente*.
5. Cria os respetivos ItemFatura.
6. Limpa o carrinho.

A fatura em PDF é gerada por FaturaController e inclui nº de encomenda, e-mail do cliente, data, método de pagamento, tabela de itens e total pago.

4.5 Validação de Dados

- As palavras-passe são validadas no RegisterController antes de serem persistidas.
- Preços e quantidades são validados com anotações de validação nas entidades.
- O stock é novamente verificado no *checkout*, antes de criar a fatura.
- As saídas *HyperText Markup Language* (HTML) são escapadas pelo Thymeleaf por omissão, prevenindo *Cross-Site Scripting* (*Cross-Site Scripting* (XSS)).

- As consultas JPA usam parâmetros nomeados, como @Param, reduzindo o risco de injeção de SQL.

4.6 Testes

Foram testados os principais fluxos da aplicação:

- **Catálogo e filtros:** pesquisa, categoria, preço e stock alteram corretamente a listagem.
- **Carrinho autenticado:** adição, remoção, quantidades, total e stock são atualizados corretamente.
- **Carrinho de visitante:** os itens permanecem na sessão e são migrados após *login* ou registo.
- **Checkout:** a compra cria fatura, cria itens, desconta stock e limpa o carrinho.
- **Fatura PDF:** o documento pode ser descarregado após a compra e no perfil.
- **Permissões:** clientes não acedem a /admin/** e visitantes não acedem a /checkout/**.
- **Conta suspensa:** uma conta suspensa não consegue iniciar sessão.
- **Encomendas:** a alteração de estado feita pelo administrador aparece no perfil do cliente.

4.7 Limitações

Apesar de funcional, a implementação apresenta algumas limitações:

- Algumas tabelas usam paginação no cliente, o que pode prejudicar desempenho com muitos registos.
- A gestão de stock não trata concorrência avançada entre compras simultâneas.
- A opção `ddl-auto=update` facilita o desenvolvimento, mas não substitui migrações controladas em produção.

4.8 Conclusões

A implementação cobre os requisitos principais do AutoUBI. O ponto mais forte é a integração entre catálogo, carrinho, *checkout*, faturas e encomendas. O ponto que mais beneficiaria de evolução é a separação da lógica de negócio em serviços dedicados.

Capítulo

5

Conclusão

A realização deste projeto permitiu aplicar conceitos de desenvolvimento web e sistemas distribuídos num cenário integrado: catálogo, carrinho, autenticação, autorização, persistência, geração de faturas e gestão de encomendas. A ligação aos conteúdos das aulas 08 a 12 foi mais evidente nas decisões sobre separação de responsabilidades, gestão de sessão, comunicação por pedidos HTTP e preocupação com segurança [1, 2, 3, 4, 5].

Considera-se que os requisitos principais foram cumpridos. O sistema suporta três perfis de utilização, permite compras com emissão de PDF, mantém histórico de encomendas e fornece ao administrador ferramentas de gestão. A funcionalidade que melhor demonstra integração entre componentes é o *checkout*, porque envolve validação de stock, persistência da fatura, atualização dos veículos e limpeza do carrinho.

Ainda assim, a solução não deve ser apresentada como pronta para produção. Existem limitações importantes: falta uma camada de serviços mais consistente, a paginação deveria passar para o servidor, o controlo de stock deveria ser reforçado para compras simultâneas e a evolução do esquema deveria usar migrações versionadas. Estas limitações não anulam o trabalho desenvolvido, mas mostram claramente onde o projeto teria de evoluir.

Como trabalho futuro, destacam-se: recuperação de palavra-passe, notificações por e-mail, avaliações de veículos, filtros por marca, ano e combustível, containerização, testes automatizados e migrações com Flyway ou Liquibase.

Bibliografia

- [1] Universidade da Beira Interior. Sistemas Distribuídos - Lecture 08: Concurrency and Threads, 2024. [Material de Apoio] <https://ttools.ubi.pt/edumark/view?filename=SD-Lecture-08.md&branch=14352-SD>.
- [2] Universidade da Beira Interior. Sistemas Distribuídos - Lecture 09, 2024. [Material de Apoio] <https://ttools.ubi.pt/edumark/view?filename=SD-Lecture-09.md&branch=14352-SD>.
- [3] Universidade da Beira Interior. Sistemas Distribuídos - Lecture 10, 2024. [Material de Apoio] <https://ttools.ubi.pt/edumark/view?filename=SD-Lecture-10.md&branch=14352-SD>.
- [4] Universidade da Beira Interior. Sistemas Distribuídos - Lecture 11, 2024. [Material de Apoio] <https://ttools.ubi.pt/edumark/view?filename=SD-Lecture-11.md&branch=14352-SD>.
- [5] Universidade da Beira Interior. Sistemas Distribuídos - Lecture 12, 2024. [Material de Apoio] <https://ttools.ubi.pt/edumark/view?filename=SD-Lecture-12.md&branch=14352-SD>.